

Lessons learned

The program was an interesting challenge to begin with. The overall structure was clear from the start, the two models needed to be made for users and push up records. Then I focused on the Http server and the Router. The Router should read the Http request, split it into the method and path, see which path is it and call the appropriate handler. Then the user controller was created to take care of the logic behind registration, login and the rest of the functions. I first used a static list of users to check the functionality of the program before making a database. For json deserialization a package was needed called `Newtonsoft.Json`. After these steps an user could register and login and the server was listening for clients. Then a `TokenManager` was made to make sure if a provided token matches the logged in user. So only authorized access was permitted. After that functions for saving pushups and getting them were made. One would save them and the duration and the other would check if the user id was the same as the current user and then get the pushup count. Next was Tournament logic. The idea is to have a 2 minute tournament and to find the others who trained, determine the winner and calculate ELO. Some other functions that were implemented were the scoreboard function and the unique feature function for achievements. There should be achievements for the first pushup entries, for doing 100 pushups in one session, for reaching 500 pushups and so on. Then the database was created with two tables users and pushup records. For this step I installed the PostgreSQL and a `Npgsql` package. The `DatabaseManager` was made for inserting and getting the data from the database. A few functions needed to be updated to go from static saving to the database. After that it was time for unit tests. I needed to install the Nunit package but there were some problems. I installed it into the main folder and then into the folder for tests because it didn't want to work then I removed it and added it just to the main folder. It still wasn't working as expected because it didn't recognize the assert functions. Then I found the solution on stack overflow. The Nunit framework needs to be included with the Nunit legacy. Then the assert functions could be used as `ClassicAssert`. Then there was the error about the duplicate versioning `cs0579` this was solved by inserting some ... Then I tried out the Integration Tests and had a problem with hanging. The program would just wait for something and never end. Then I realized that I implemented the `readrequestbody(reader)` function at multiple places in a file where I was handling requests. So after reading the body the program would try to read it again but nothing was coming this time around. This was solved by deleting the unnecessary requests. The given curl file for testing was problematic as well. The port needed to be changed to 10001 because I previously had 8080. After that the testing of different functions was problematic because the name of routes in the Router file was different. But after adapting the program it has worked. Then I discovered the problem with tokens. No tokens were saved because the name of a column differed from the sql query and the table column. sql query had the duration and the table had `duration_seconds`. After that was repaired the tokens were saved. Some explanations for the unit test: `NewUser_ShouldHaveDefaultElo1000`. It confirms that all new users start fairly. Every user must start equally (at 1000). If this default is wrong, the entire ranking system is unfair or inconsistent. `UserAchievements_ShouldStartEmpty`. Ensures no achievements are pre-assigned. Achievements are earned. This test prevents bugs where a user appears to have unlocked things they haven't earned yet. `Token_ShouldBeUnique`. Validates that each login gets a fresh, unique token. Token uniqueness ensures secure and correct user authentication. If two users share the same token by accident, it creates major security and data issues. `Elo_ShouldIncrease_ForWinner`. Simulates a tournament win. The game logic relies on fair reward. Without this test, winners might not receive ELO gain — breaking the

competition system.. The time spent on this project was around 50 hours spread across a few days.

Link the the git page : <https://github.com/alekskrz/SEB.git>